

Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models
GitHub Repository	https://github.com/sangee-3010/Machine_Learning
Student Name	Sangeetha S K
Register Number	3122247001054
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	03/08/2026

1 Objective

To implement and analyze Logistic Regression and Support Vector Machine (SVM) models for spam email detection, and to evaluate how kernel selection and parameter tuning affect the final classification performance.

2 Problem Statement

The Spambase dataset consists of numerical attributes derived from email messages, including word occurrence frequencies, character frequencies, and capital letter sequence statistics. The objective of this experiment is to develop binary classification models that can distinguish between spam emails (class 1) and legitimate emails (class 0). Each email is represented using a 57-feature numerical vector, which is provided as input to the classifiers. Logistic Regression and Support Vector Machine models with different kernel functions are trained, optimized, and compared to determine the model that provides the best trade-off between classification performance, generalization ability, and computational efficiency.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	Kaggle (https://www.kaggle.com/datasets/somesh24/spambase)
Number of Samples	4601
Number of Features	57
Number of Classes	2 (Spam = 1, Ham = 0)
Missing Values	None (0 missing values across all 58 columns)
Train-Test Split	80% Train / 20% Test (<code>train_test_split, random_state=42</code>)

4 Exploratory Data Analysis

Exploratory analysis was performed to examine class distribution, feature relationships, and feature behaviour before applying classification algorithms.

Class Distribution

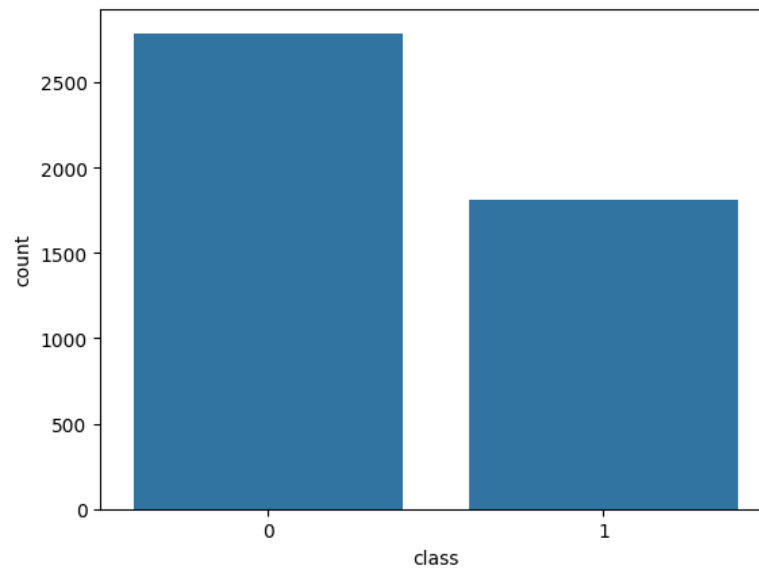


Figure 1: Class Distribution of Spam (1) vs Ham (0)

Inference:

The bar chart shows the number of emails belonging to each class, where class 0 (ham) clearly outnumbers class 1 (spam) in the dataset. Based on the target mean of 0.394045, approximately 60.6% of emails are ham and 39.4% are spam. This represents a moderate class imbalance rather than a severe one. It is not extreme enough to require resampling techniques such as SMOTE, but it does justify reporting Precision, Recall, and F1-score alongside Accuracy, since accuracy alone can be misleading when classes are imbalanced. This distribution also informed the choice to inspect precision and recall separately for the spam class during evaluation.

Correlation Matrix

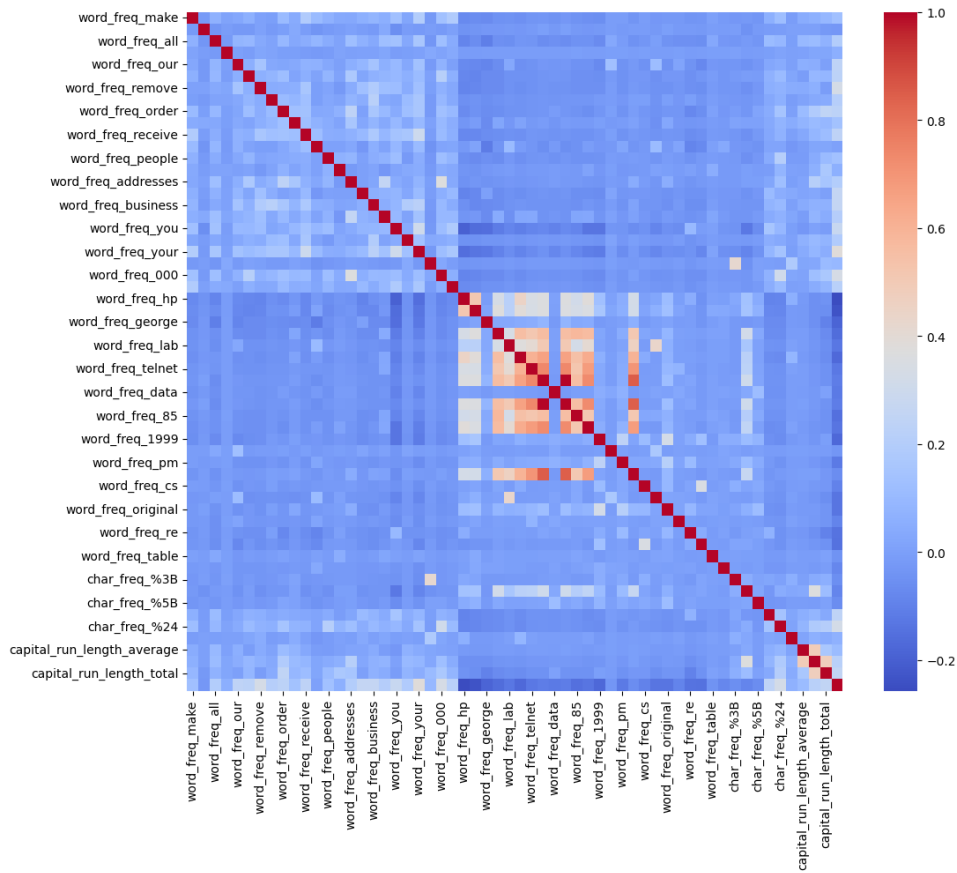


Figure 2: Correlation Heatmap of Dataset Features

Inference:

The heatmap visualizes pairwise Pearson correlation between all 57 numerical features using a coolwarm color scale, where dark red indicates strong positive correlation. Most of the word-frequency features exhibit weak correlation with one another, appearing as a mostly blue/neutral region across the majority of the matrix. In contrast, the three capital-run-length features (`capital_run_length_average`, `longest`, and `total`) and a cluster of technical-term word frequencies (e.g., `hp`, `lab`, `telnet`, `data`, `85`, `cs`) form a visibly brighter block, indicating stronger mutual correlation. The generally low correlation among most features suggests limited multicollinearity, which is favorable for linear models like Logistic Regression.

Feature Distribution

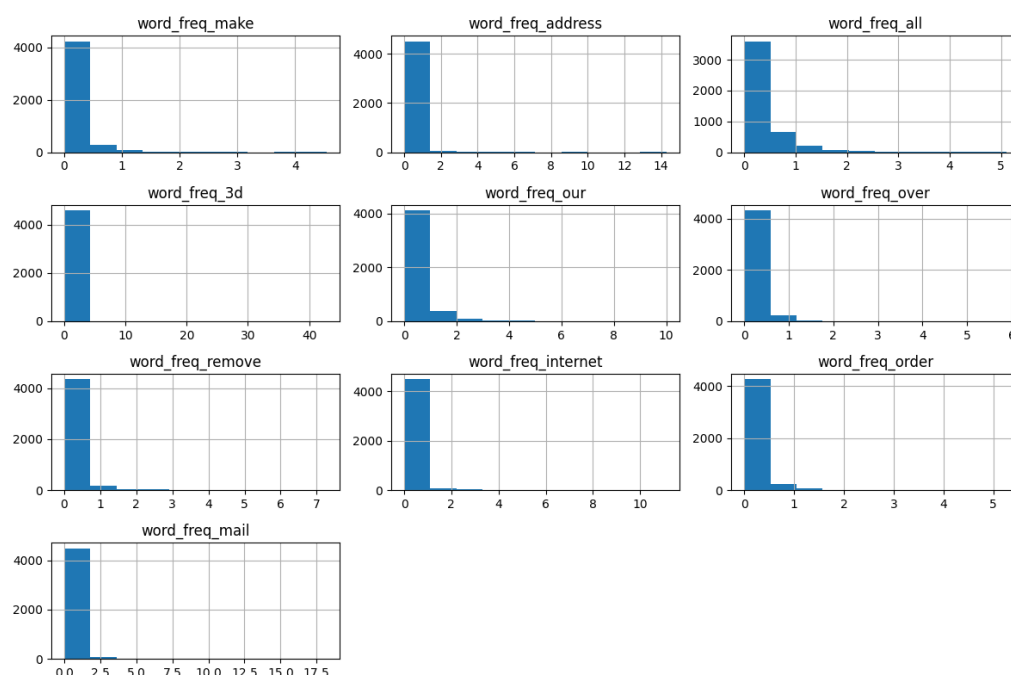


Figure 3: Histograms of the First 10 Features

Inference:

The histograms for the first 10 word-frequency features (`word_freq_make` through `word_freq_mail`) all show a strongly right-skewed distribution, with the overwhelming majority of values concentrated at or near zero and a long thin tail extending toward higher frequencies. This indicates that most emails do not contain a given word at all, while only a small fraction contain it repeatedly. Such skewed, sparse distributions are typical of word-frequency data and motivated the use of feature standardization before model training. This skew also explains why several features show extreme outlier values in the corresponding boxplot.

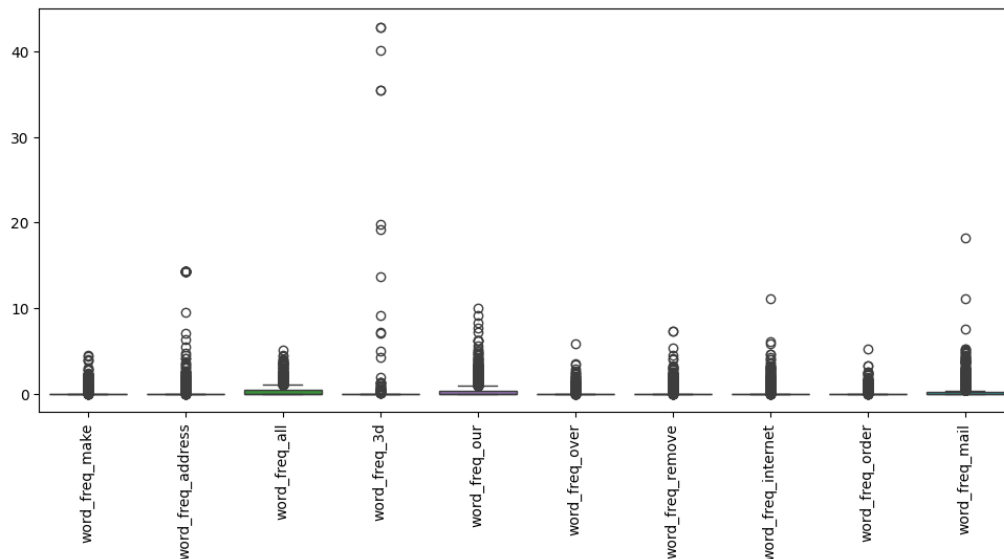


Figure 4: Boxplot of the First 10 Features

Inference:

The boxplot of the same 10 features shows medians and interquartile ranges compressed close to zero for nearly every feature, with a large number of points plotted well above the upper whisker, particularly for `word_freq_3d`, `word_freq_our`, and `word_freq_mail`. This confirms the presence of numerous outlier-like high-frequency values consistent with the skewed histograms. These outliers are legitimate data points (emails that repeat a word many times) rather than data errors, so no outlier removal was performed; instead, standardization was used to reduce the influence of scale differences across features.

5 Data Preprocessing

- **Missing Value Handling:** The dataset was verified to have zero missing values across all 58 columns using `df.isnull().sum()`, so no imputation was required.
- **Encoding:** All 57 input features are already numeric (word/character frequencies and capital-run-length statistics), and the target `class` column is already binary (0/1), so no categorical encoding was necessary.
- **Feature Scaling:** All input features were standardized using `StandardScaler`, fit on the training data and applied to both the training and test sets, to ensure features with different scales (e.g., frequency percentages vs. capital-run-length counts) contribute comparably to distance- and margin-based algorithms such as SVM.
- **Train-Test Split:** The dataset was split into 80% training data and 20% testing data using `train_test_split` with `random_state=42`, giving 3680 training samples and 921 test samples (531 ham, 390 spam).
- **Feature Engineering:** No additional feature engineering (e.g., polynomial features or feature selection) was performed; all 57 original features were used directly.

6 Machine Learning Algorithms

Logistic Regression

Working Principle: Logistic Regression is a probabilistic linear classifier that models the probability of an email being spam using the sigmoid function applied to a weighted sum of the input features. A decision threshold (0.5 by default) converts this probability into a binary class label.

Mathematical Formulation:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

The model is trained by minimizing the log-loss (cross-entropy loss), optionally with an L1 (Lasso) or L2 (Ridge) regularization penalty controlled by the inverse regularization strength C .

Advantages: Computationally efficient, highly interpretable (feature weights indicate influence and direction), performs well when the classes are close to linearly separable, and requires minimal tuning.

Limitations: Assumes an approximately linear decision boundary between classes, and can underperform relative to non-linear models such as kernel SVMs when the true decision boundary is more complex.

Support Vector Machine (SVM)

Working Principle: SVM is a margin-based classifier that finds the optimal hyperplane separating the two classes by maximizing the margin between them. Only a subset of training points (the support vectors) determine the position of this boundary. Using kernel functions (linear, polynomial, RBF, sigmoid), SVM can also model non-linear decision boundaries by implicitly mapping data into a higher-dimensional space.

Mathematical Formulation: For a kernel function $K(x_i, x_j)$, the SVM decision function is

$$f(x) = \text{sign} \left(\sum_i \alpha_i y_i K(x_i, x) + b \right)$$

where α_i are learned dual coefficients. The RBF kernel used here is defined as $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$.

Advantages: Effective in high-dimensional feature spaces, capable of modeling non-linear boundaries via the kernel trick, and robust to outliers when the margin/regularization parameter C is tuned appropriately.

Limitations: Computationally more expensive to train (especially during grid search) than Logistic Regression, less interpretable due to the implicit feature mapping, and performance is sensitive to the choice of kernel and hyperparameters (C , γ , degree).

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	As per notebook environment (Google Colab)
IDE	Jupyter Notebook (Google Colab)
Operating System	Linux (Colab runtime)
Hardware	Colab standard CPU runtime

8 Hyperparameter Selection

Logistic Regression – Search Space

Parameter	Values Searched
Penalty	L1, L2
C	0.01, 0.1, 1, 10, 100
Solver	liblinear, saga

Best Parameters Found: `{'C': 10, 'penalty': 'l1', 'solver': 'liblinear'}`
(Best CV Accuracy: 0.9274)

SVM – Search Space

Parameter	Values Searched
Kernel	Linear, Polynomial, RBF, Sigmoid
C	0.1, 1, 10, 100
Gamma	scale, auto
Degree (Polynomial)	2, 3, 4

Best Parameters Found: `{'C': 1, 'degree': 2, 'gamma': 'scale', 'kernel': 'rbf'}` (Best CV Accuracy: 0.9307)

Both searches used `GridSearchCV` with 5-fold cross-validation, optimizing for accuracy.

9 Results

Logistic Regression (Tuned)

Metric	Value
Accuracy	0.917481
Precision	0.919786
Recall	0.882051
F1-score	0.900524
Training Time (s)	528.176508

SVM (Tuned, RBF Kernel)

Metric	Value
Accuracy	0.934853
Precision	0.950820
Recall	0.892308
F1-score	0.920635
Training Time (s)	796.104414

SVM – Kernel-wise Comparison (Default Hyperparameters)

Kernel	Accuracy	F1-Score	Training Time (s)
Linear	0.926167	0.910526	0.676201
Polynomial	0.766558	0.633731	0.559421
RBF	0.934853	0.920635	0.285086
Sigmoid	0.876221	0.850000	0.307000

5-Fold Cross-Validation Results

Fold	Logistic Regression	SVM
Fold 1	0.937500	0.934783
Fold 2	0.904891	0.904891
Fold 3	0.933424	0.940217
Fold 4	0.930707	0.937500
Fold 5	0.930707	0.936141
Average	0.927446	0.930707

Comparative Analysis

Criterion	Logistic Regression	SVM
Accuracy	0.917481	0.934853
Model Complexity	Low	High
Training Time (s)	528.176508	796.104414
Interpretability	High	Low

10 Result Visualization

Confusion Matrix – Baseline Logistic Regression

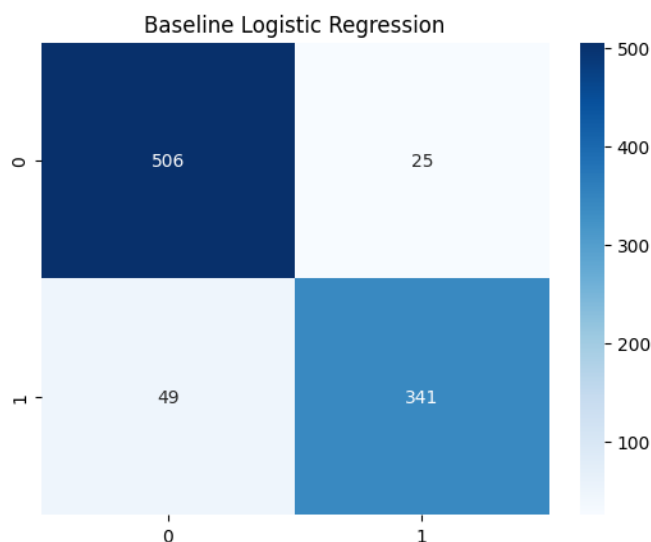


Figure 5: Confusion Matrix – Baseline Logistic Regression

Inference:

The baseline Logistic Regression model (default L2 penalty, `max_iter=5000`) correctly classified 506 ham emails and 341 spam emails out of 921 test samples, while misclassifying 25 ham emails as spam and 49 spam emails as ham. This gives an accuracy of 0.9197, with recall (0.8744) noticeably lower than precision (0.9317), showing the baseline model is somewhat more likely to miss spam emails (false negatives) than to incorrectly flag legitimate mail. This baseline serves as the reference point against which the effect of hyperparameter tuning is measured.

Confusion Matrix – Tuned Logistic Regression

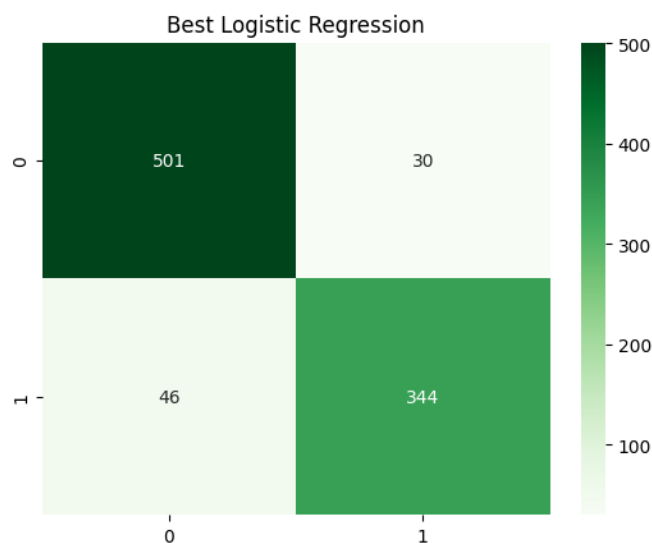


Figure 6: Confusion Matrix – Tuned (Best) Logistic Regression

Inference:

The grid-search-tuned Logistic Regression model (L1 penalty, $C=10$, liblinear solver) correctly classified 501 ham emails and 344 spam emails, with 30 false positives and 46 false negatives. Compared to the baseline, the tuned model correctly identifies 3 more spam emails (recall improves from 0.8744 to 0.8821) at the cost of 5 additional false positives, and its overall test accuracy (0.9175) is marginally lower than the baseline (0.9197). This shows that the L1-regularized, weakly-regularized (high C) configuration shifts the precision-recall balance rather than improving all metrics uniformly.

Confusion Matrix – Tuned SVM

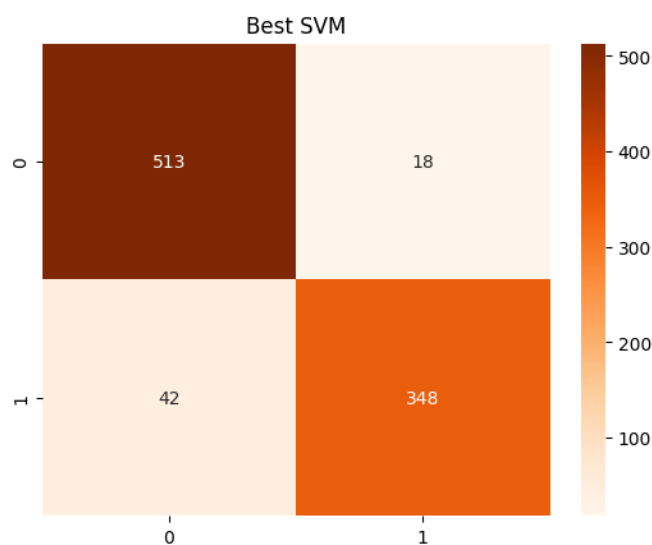


Figure 7: Confusion Matrix – Tuned (Best) SVM

Inference:

The grid-search-tuned SVM (RBF kernel, $C=1$, $\gamma=\text{'scale'}$) correctly classified 513 ham emails and 348 spam emails, with only 18 false positives and 42 false negatives – the fewest total misclassifications (60) among all three models evaluated. Its high precision (0.9508) shows it is conservative about labeling legitimate mail as spam, which is a desirable property for a spam filter since false positives (blocking real email) are typically more costly than false negatives. This confirms the RBF-kernel SVM as the best-performing model in this experiment.

11 Discussion

The experimental results show that the tuned RBF-based SVM classifier achieved better classification performance compared to the tuned Logistic Regression model. SVM obtained higher accuracy (0.9349), precision (0.9508), recall (0.8923), and F1-score (0.9206) compared to Logistic Regression, which achieved an accuracy of 0.9175 and an F1-score of 0.9005. The improved performance indicates that the non-linear decision boundary created by the RBF kernel was able to capture complex patterns present in the Spambase feature space more effectively.

The comparison of different SVM kernels also highlights the importance of kernel selection. The RBF kernel produced the best results with an accuracy of 0.9349, followed by the Linear kernel with 0.9262 accuracy. The Polynomial and Sigmoid kernels showed comparatively lower performance, indicating that these kernels were less suitable for representing the underlying relationship between email features and spam labels.

The tuned Logistic Regression model selected L1 regularization with $C=10$, indicating weaker regularization compared to smaller C values tested during the search. This configuration improved the model's ability to identify spam emails while maintaining reasonable generalization. However, compared to SVM, Logistic Regression was limited by its linear decision boundary and was unable to capture more complex relationships between features.

In terms of computational efficiency, Logistic Regression required less training time (528.2 seconds) compared to SVM (796.1 seconds) during hyperparameter tuning. This demonstrates the trade-off between computational cost and predictive performance, where SVM provides better classification results at the expense of higher training complexity.

Overall, the experiment demonstrates that kernel-based methods are effective for this binary classification problem, with RBF-SVM providing the best balance of accuracy and generalization performance. Future improvements can include applying class-weight optimization, feature selection techniques, and additional evaluation methods such as ROC-AUC and Precision-Recall analysis to further improve and analyze the classifier performance.

12 Conclusion

This experiment implemented and evaluated Logistic Regression and Support Vector Machine classifiers for binary spam email classification using the Spambase dataset. A complete preprocessing pipeline was applied, including feature standardization and train-test splitting, followed by hyperparameter tuning using GridSearchCV with 5-fold cross-validation.

Among the evaluated models, the tuned SVM with an RBF kernel achieved the best overall performance with the highest test accuracy, precision, recall, F1-score, and cross-validation accuracy. The RBF kernel effectively captured the non-linear patterns present in the email feature space, allowing SVM to outperform the linear Logistic Regression model.

Although Logistic Regression provided better interpretability and lower computational complexity, SVM achieved superior predictive performance at the cost of increased training time. The experiment demonstrates the trade-off between model simplicity and classification accuracy, where linear models offer efficiency and transparency while kernel-based models provide greater flexibility for complex decision boundaries.

Overall, the results indicate that the RBF-kernel SVM is the most suitable model for this dataset. Future improvements can be achieved through feature selection, class-weight optimization, threshold tuning, and evaluation using additional metrics such as ROC-AUC and Precision-Recall curves.

13 References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. Available: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
- [2] C. Cortes and V. Vapnik, “Support-Vector Networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995. Documentation: <https://scikit-learn.org/stable/modules/svm.html>
- [3] Scikit-learn Developers, “Tuning the hyper-parameters of an estimator,” Scikit-learn Documentation. Available: https://scikit-learn.org/stable/modules/grid_search.html
- [4] Somesh24, “Spambase Dataset,” Kaggle. Available: <https://www.kaggle.com/datasets/somesh24/spambase>
- [5] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, 1999.